Proceedings of the 2001 IEEE
International Conference on Robotics & Automation
Seoul, Korea · May 21-26, 2001

Evaluation of Path Length Made in Sensor-Based Path-Planning with the Alternative Following

Yohei Horiuchi and Hiroshi Noborio
Division of Information and Computer Science, Graduate School of Engineering
Osaka Electro-Communication University
Hatsu-Cho 18-8, Neyagawa, Osaka 572-8530, Japan

Abstract

As the sensor-based path-planning algorithm whose upper bound of path length is the smallest, Alg1 and Alg2 have been well known. In these algorithms, after encountering a visited point (that is, enters into a loop), a mobile robot follows an uncertain obstacle by an opposite direction until it can leave the obstacle. We call this as the "reverse procedure". Independently, if a robot always changes a direction following an uncertain obstacle alternatively, the robot arrives at a destination earlier on average (a probability for the robot to join a loop around a destination decreases). We call this as the "alternative following". In this paper, by mixing the reverse procedure and alternative following, we design new sensor-based path-planning algorithms Rev1 and Rev2.

The upper bound $2D + 2\Sigma_i P_i$ in Rev1 and Rev2 is slightly longer than the upper bound $D + 2\Sigma_i P_i$ in Alg1 and Alg2 (P_i: the perimeter of an i-th encountered obstacle, D: The Euclidean distance between start and goal points). However, the average bound of path lengths in Rev1 and Rev2 will be shorter than that in Alg1 and Alg2. The former is fortunately evaluated by mathematical proofs, but the latter is unfortunately ascertained by several simulation results.

1 Introduction

In the past decade, many sensor-based path-planning algorithms have been proposed [6],[11]. Almost all the algorithms have been designed for selecting a shorter path until a destination. In the historical order, Lumelsky initially proposed *Bug1* and *Bug2* whose upper bounds of path lengths are denoted by $D+1.5\Sigma_i P_i$ and $D+\Sigma_i \frac{n_i}{2} P_i$, respectively [1] ($D$: the Euclidean distance between start and goal points, P_i: the perimeter of an i-th encountered obstacle, n_i: the number of encountering an i-th obstacle). However, many people guess that the latter algorithm selects a shorter collision-free path until a destination than the former algorithm on average. The reason is that an algorithm whose upper bound is the smallest is not always equivalent to an algorithm whose average bound is the smallest. Therefore, *Bug2* has been repeatedly modified.

For example, Sankaranarayanan modified *Bug2* as *Alg1* whose worst length is $D + 2\Sigma_i P_i$ by adding the "reverse procedure" [2]. If a robot enters into a loop (meets a visited point), the robot goes back to the last hit point and reverses the direction following the obstacle. By this procedure, *Alg1* eliminates a repetitive loop (i.e., a side trip to a destination). On the other hand, one of the authors deeply investigated convergence of a robot to a destination theoretically as the metric condition, and proposed *Class1* whose worst length is bounded [3],[5]. Then, Sankaranarayanan modified *Class1* as *Alg2* whose worst length is $D + 2\Sigma_i P_i$ by adding the "reverse procedure" [4].

In order to shorten path length on the average in an unknown environment, one of the authors proposed *Bug2(alter.)* and *Class1(alter.)* (their originals are called *Bug2(const.)* and *Class1(const.)*) [7]. The difference is for a robot to select alternatively clockwise and counter-clockwise directions following an uncertain obstacle. By the "alternative following", a robot quickly arrives at a destination, whose property is theoretically ensured on average.

If and only if there is an unique obstacle in a 2-D uncertain environment, the "reverse procedure" and "alternative following" are smartly mixed as the sensor-based path-planning algorithm *Single*. The upper bound of path length is evaluated by $2D + P$ [10]. Otherwise, we could not get any smart algorithm. To overcome this, we investigate how to mix the "reverse procedure" and "alternative following" and consequently propose two sensor-based path-planning algorithms *Rev1* and *Rev2* whose upper bound of path length is $2D + 2\Sigma_i P_i$. Furthermore, by several simulation results, we indicate that paths selected in *Rev1* and *Rev2* is extremely shorter than those in *Alg1* and *Alg2*.

This paper is organized as follows: Section 2 describes a robot and its unknown environment. Then, we explain two sensor-based path-planning algorithms

0-7803-6475-9/01/$10.00© 2001 IEEE

*Rev*1 and *Rev*2. In section 3, we give mathematical proofs which *Rev*1 and *Rev*2 have an upper bound $2D + 2\Sigma_i P_i$ of their path lengths. In section 4, we conclude path lengths in *Rev*1 and *Rev*2 is extremely shorter than those in *Alg*1 and *Alg*2 on average by several simulation results. Finally, section 5 presents a few concluding remarks.

2 The algorithms Rev1 and Rev2

In this section, we explain proposed sensor-based path-planning algorithms *Rev*1 and *Rev*2. In the algorithms, a robot is represented as a point and can move omnidirectionally in a 2-D uncertain environment. The environment has many unknown obstacles. The number of obstacles or the perimeter of each obstacle is of finite. Both consist of normal and reverse procedures. In the normal procedure, a mobile robot goes straight to its destination and also follows boundary of an uncertain obstacle for the first time. In the reverse procedure, the robot follows a route for the second time, which was already traversed.

2.1 The algorithm Rev1

In this paragraph, we explain a proposed algorithm *Rev*1. This is designed under *Alg*1 with the alternative following, and also *Alg*1 is designed under *Bug*2 with the reverse procedure [3],[5],[2],[7].

In *Rev*1, an initial point, a target point, a present point are denoted by S, G, R, respectively. In addition, hit and leave points around uncertain obstacles are denoted by H_i and L_i (i=1,2,...), respectively. Also, a sequence of candidate points H_i (i=1,2,...) to expand is completely stored in *Hlist*. Moreover, a direct path without any loop from S to R is always memorized into *Plist*, which is composed of routes made in the normal procedure, and its driving distance is calculated as DP. In general, *Hlist* is used for selecting a hit point so as to escape from a local loop, and *Plist* is used for picking up a proper route until the selected hit point. Moreover, directions $Dir(H_i)$ and $Dir(L_i)$ to follow an obstacle are prepared at hit and leave points H_i and L_i, respectively. Finally, i=1, $S = L_0 = R$, Dir=1 [1: clockwise direction, -1: counter-clockwise direction] are initialized.

1. A robot R goes straight to a goal G until the following occurs:

(1a) If a robot arrives at a goal G, exit with success.

(1b) If a robot encounters an uncertain obstacle, set a hit point H_i by a robot R, $DP(H_i)$=DP, and $Dir(H_i) = Dir$. Then, register H_i into *Hlist*, memorize H_i and $Dir(H_i)$ into *Plist*, and then go to step 2.

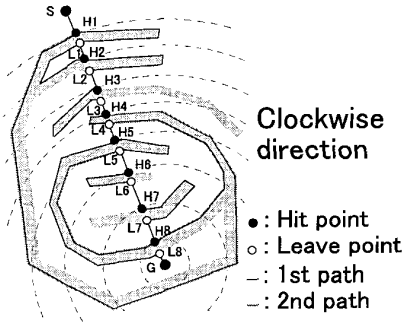

Figure 1 : A collision-free path between S and G via many leave and hit points (L_i and H_i) selected in *Rev*1.

2. Dir is checked at H_i in *Hlist*, and if both (clockwise and counter-clockwise) directions were already checked at H_i, it is immediately eliminated in *Hlist*. Then, a robot faithfully traces an obstacle by the direction Dir until the following occurs:

(2a) If a robot R arrives at a goal G, exit with success.

(2b) If the Euclidean distance $\mid RG \mid$ is smaller than the shortest distance E and if the robot R is to be on the segment SG, E is replaced by $\mid RG \mid$. They are called the *metric* and *segment* conditions. Furthermore, if a robot R can go straight to a goal G, regard a leave point L_i by a robot R. This is called the *physical* condition. Then, set $DP(L_i) = DP$, $Dir = Dir * -1$, and $Dir(L_i) = Dir$. In addition, register L_i and $Dir(L_i)$ into *Plist*. Finally, $i = i+1$, and back to step **1**. Otherwise, this step is continued.

(2c) If a robot R returns to the last hit point H_i, exit with failure. In this case, a goal G is completely enclosed by obstacle boundary and therefore there is no deadlock-free path until the goal G.

(2d) If a robot R returns to a past hit point H_k, the former point H_k is memorized as the same later point Q_l into *Hlist*. Then, the robot returns to the hit point H_j booked in *Hlist*, which is the closest to G. In this case, H_j always corresponds to H_i. Then, the robot selects a shorter route of clockwise and counter-clockwise routes already traversed between Q_l and H_j. That is, if $d_1 \geq d_2$ ($d_1 = DP(H_j) - DP(H_k)$ and $d_2 = DP(Q_l) - DP(H_j)$), a robot R moves from Q_l to H_j using $Dir(L_n)(l \geq n \geq j)$ in *Plist*, otherwise, a robot R moves from H_k to H_j using $Dir(H_n)(k \leq n \leq j)$ in *Plist*. In both cases, a sequence of hit and leave points between Q_l and H_j is eliminated in *Plist*, and also set $DP = DP(H_j)$ and the direction Dir is selected as -1**Dir*(H_j), which has not been tested at H_j. Finally, this step is continued.

(2e) If a robot R returns to a past leave point L_k, the former point L_k is memorized as the same later point Q_l into *Hlist* and a present direction Dir is checked at $H_n(k \leq n \leq i)$ in *Hlist*. Moreover,

if both (clockwise and counter-clockwise) directions have been already checked at a hit point H_n, H_n is removed from *Hlist*. Then, a robot returns to a hit point H_j booked in *Hlist*, which is the closest to G. R returns to H_j by a shorter route of clockwise and counter-clockwise routes from Q_l and L_k to H_j, respectively. That is, if $d_1 \geq d_2$ ($d_1 = DP(H_j) - DP(L_k)$ and $d_2 = DP(Q_l) - DP(H_j)$), a robot R moves from Q_l to H_j using $Dir(L_n)(l \geq n \geq j)$ in *Plist*, otherwise, a robot R moves from L_k to H_j using $Dir(H_n)(k \leq n \leq j)$ in *Plist*. In both cases, a sequence of hit and leave points between Q_l and H_j is eliminated in *Plist*, and also set $DP = DP(H_j)$ and the direction Dir is selected as -1*$Dir(H_j)$, which has not been tested at H_j. Finally, this step is continued.

In Fig.1, details of our algorithm is explained. After checking the counter-clockwise direction at H_2, a mobile robot enters into a counter-clockwise loop at L_1. Then, the robot returns to H_2 by a shorter route, i.e., the straight segment L_1H_2. At this time, a direct route $S - H_1 - L_1 - H_2$ is memorized into *Plist*. Then, a robot follows an obstacle by the clockwise direction at H_2. Since both directions were checked, H_2 is removed from *Hlist*.

After selecting the clockwise direction at H_4, a robot joins a global loop at H_1. Therefore, the robot returns to H_4 by a shorter route $H_1 - L_1 \sim -H_4$, and then selects the counter-clockwise direction at H_4. Therefore, H_4 is removed from *Hlist* because both directions were checked. Here, a direct route $S - H_1 - L_1 \sim -H_4$ is memorized into *Plist*. Then, the robot enters into a counter-clockwise loop at L_5. Therefore, the counter-clockwise direction is checked at H_6, H_7 and H_8. Since both directions were tested, H_7 is removed from *Hlist*. Now, H_1, H_3, H_5, H_6, H_8 are located in *Hlist*. Since H_8 is the closest to G, the robot returns to H_8. In order to return to H_8, a robot compares a counter-clockwise route $L_5 - H_6 - L_6 - H_7 - L_7 - H_8$ with a clockwise route $L_5 - H_8$. In this case, it selects the clockwise route as a shorter route. At this time, a direct route $S - H_1 \sim -L_7 - H_8$ is memorized into *Plist*.

After this, the robot selects the clockwise direction at H_8, and consequently H_8 is removed from *Hlist*. Then, the robot enters into a clockwise loop at L_4. For this reason, the clockwise direction is checked at H_5 and H_6. Since both directions were checked, H_6 is removed from *Hlist*. Now, H_1, H_3, and H_5 are located in *Hlist*, and therefore H_5 is the closest to G. In order to go to H_5, a robot compares a counter-clockwise route $L_4 - H_8 - L_7 - H_7 - L_6 - H_6 - L_5 - H_5$ with a clockwise route $L_4 - H_5$. In this case, it selects the clockwise route as a shorter route. At this time, a direct route $S - H_1 \sim -L_4 - H_5$ is memorized into *Plist*.

Finally after starting from H_5 by the counter-clockwise direction, the robot arrives at G.

2.2 The algorithm Rev2

In this paragraph, we explain a proposed algorithm *Rev*2. This is designed under *Alg*2 with the alternative following, and also *Alg*2 is designed under *Class*1 with the reverse procedure [3],[5],[4],[7]. Here, we describe the algorithm *Rev*2 by changing the procedure **(2b)** in the algorithm *Rev*1.

(2b) If the Euclidean distance | RG | is smaller than the shortest distance E, E is replaced by | RG |. Furthermore, if a robot R can go straight to a goal G, regard a leave point L_i by a robot R. This is called the *physical* condition. In addition, register L_i into *Plist*. Then, set $DP(L_i) = DP$. In addition, $Dir = Dir * -1$, $Dir(L_i) = Dir$. $i = i + 1$. Finally, back to step 1. Otherwise, this step is continued.

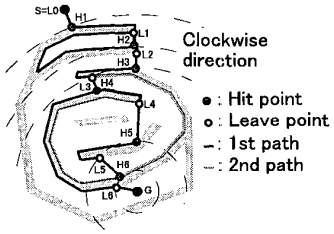

Figure 2 : A collision-free path between S and G via many leave and hit points (L_i and H_i) selected in *Rev*2.

Finally, we briefly conclude *Rev*1 and *Rev*2 (Figures 1 and 2). These algorithms *Rev*1 and *Rev*2 are designed under *Alg*1 and *Alg*2 including the alternative following. If a mobile robot encounters a previous leave or hit point, the robot returns to a booked hit point which is the closest to a destination. Here, if a robot returns to a leave point L_k by the clockwise [counter-clockwise] following, the following is prohibited at a set of hit points on an interval H_iL_k. The reason is that the set is completely enclosed by a local loop around obstacle boundary. Then, if both directions at an arbitrary hit point H_j are prohibited, the hit point H_j is removed from *Hlist*. On the other hand, even though a robot returns to a hit point H_k, it is enclosed by a global loop (including at least one crack among obstacles). For this reason, prohibition and elimination of the hit point are not considered.

3 Theoretical Proofs

First of all, we explain the metric condition of *Rev*1 and *Rev*2 as follows: A mobile robot R finally arrives at a destination G if the robot leaves an encountered obstacle at a point L_i whose distance toward G is smaller than the shortest distance CG in *Rev*2 (on the segment SG in *Rev*1). Based on this condition, a mobile robot cannot meet a path, which

was already traced from a start S except a set of leave and hit points.

Theorem 1 A mobile robot never encounters a set of traced obstacle boundaries and straight segments except a set of hit and leave points.

Proof: A mobile robot R always renews the closest point C everywhere in $Rev2$ or C on the segment SG in $Rev1$. Here, we assume that the robot encounters a traced obstacle boundary or a traced straight segment at a point A (Fig.3). Because of the metric condition, L_i is closer to G than A, and A is closer to G than L_i in Fig.3(a). In addition, H_k is closer to G than A, L_{i-1} is closer to G than H_k, and A is closer to G than L_{i-1} in Fig.3(b). By these contradictions, the point A never appears on traced obstacle boundary or traced straight segment in $Rev1$ and $Rev2$. Q.E.D.

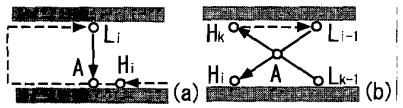

Figure 3 : (a) A robot never encounters an arbitrary point A on a traced obstacle boundary in $Rev2$ (A on SG in $Rev1$). (b) A robot never passes through a traced straight segment at an arbitrary point A in $Rev2$ (at A on SG in $Rev1$).

Based on the theorem 1, a mobile robot can encounter only a set of hit and leave points. If a robot encounters such a visited point L_k or H_k by a clockwise [counter-clockwise] direction, the normal procedure is switched into the reverse procedure. Therefore in the normal procedure, a mobile robot traces obstacle boundary or straight segment once, i.e., a coefficient of D or $\Sigma_i P_i$ is one. In the reverse procedure, a mobile robot firstly seeks for a booked hit point H_j which is the closest to a destination G, and secondly goes back to H_j along a shorter route of clockwise and counter-clockwise routes between L_k or H_k and H_j. Also in the reverse procedure, a mobile robot traces obstacle boundary or straight segment once, i.e., a coefficient of D or $\Sigma_i P_i$ is one. Therefore, we summarize that a coefficient of D or $\Sigma_i P_i$ is totally bounded by two. In this section, this characteristic is theoretically described.

In almost all the previous algorithms, a mobile robot enters into global and local loops several times in an unknown maze. Consequently, the robot generates a very long path until a destination.

Definition: If a mobile robot encounters a hit point, the robot enters into a global loop. A robot revolves round initial and target points (S and G) along a global loop (Fig.4(a),(b)). On the other hand, if a mobile robot encounters a leave point, the robot enters into a local loop. A robot does not go round initial and target points (S and G) along a local loop (Fig.4(c)). If and only if a mobile robot traces boundary of an encountered obstacle by a fixed (clockwise or counter-clockwise) direction, the robot sometimes enters into a global loop but never joins a local loop. This is trivial. Otherwise, a mobile robot sometimes enters into global and local loops. ○

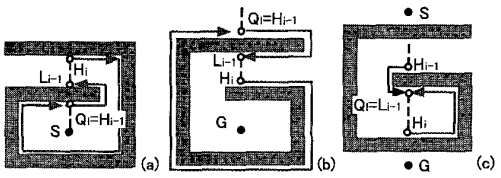

Figure 4 : (a) A global loop appears around an initial point S if a robot returns to a hit point $Q_l = H_{i-1}$. (b) A global loop appears around a target point G if a robot returns to a hit point $Q_l = H_{i-1}$. (c) A local loop without S and G appears if a robot returns to a leave point $Q_l = L_{i-1}$.

For example, the algorithms $Bug2$ and $Class1$ does not make any local loop since they fix a direction following an encountered obstacle. However, since they make a lot of similar global loops repeatedly, they pick up very long collision-free paths in an uncertain environment. To overcome this problem, we have two techniques as follows:

(1) After finding a global or local loop, we reverse a following direction ($Alg1$ [2] and $Alg2$ [4]): If a robot encounters a hit or leave point Q_l (i.e., meets a global or local loop), a robot returns to the last hit point H_i by a shorter route between Q_l and H_i, and then reverses its direction following an obstacle (i.e., escapes from a global or local loop). We call this as the "reverse procedure". By this, a robot traces obstacle boundary at most twice and goes along straight segment at most once. As a result, an upper bound of path length in $Alg1$ and $Alg2$ is estimated as $D + 2\Sigma_i P_i$.

(2) Before finding a global or local loop, we change a following direction alternatively ($Bug2(alter.)$ and $Class1(alter.)$ [7]): If a robot alternatively changes a direction following an obstacle, a probability that a robot joins a global loop decreases. Unfortunately, we could not estimate any number of tracing obstacle boundary or going straight segment. As a result, an upper bound of path length in $Bug2(alter.)$ and $Class1(alter.)$ is not estimated at all.

Furthermore in our sensor-based path-planning algorithm $Single$, we succeeded in mixing advantages of the algorithms $Alg1$ and $Bug2(alter.)$ or $Alg2$ and $Class1(alter.)$ in an uncertain maze whose collision-free path to a destination (and needless to say, its obstacle) is unique [10]. An upper bound of path length in $Single$ is regarded as $2D + P$.

In this paper, we propose $Rev1$ and $Rev2$ by mixing advantages of $Alg1$ and $Bug2(alter.)$ or $Alg2$ and $Class1(alter.)$ in an uncertain maze with two or more collision-free paths to a destination. The purpose of this section is to evaluate an upper bound of path lengths in $Rev1$ and $Rev2$ as $2D + 2\Sigma_i P_i$.

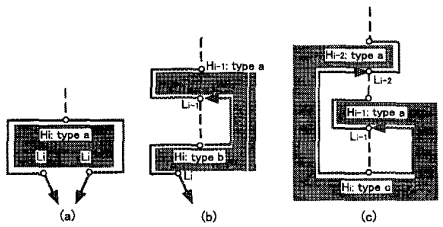

Figure 5 : (a) A robot always finds different leave points L_i from a hit point H_i by the clockwise and counter-clockwise directions. (b) A robot cannot find any leave point from a hit point H_i by the clockwise direction. (c) A robot cannot find any leave point from a hit point H_i by both directions.

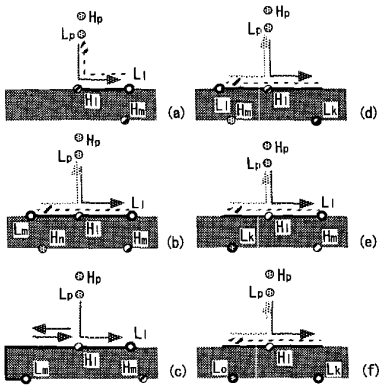

Figure 6 : (a),(b),(c),(d),(e),(f) Six possible cases (a-1), (a-2), (a-3), (b-1), (b-2) and (c-1) where a mobile robot traces obstacle boundary or straight segment. Each obstacle means a part of a whole obstacle around a hit point H_l, whose shape is free.

In both algorithms, a mobile robot sequentially hits and leaves a set of uncertain obstacles at H_i and L_i, respectively. Therefore, there is a sequence of H_i and $L_i (i = 1, ..., n-1)$ in an uncertain environment ($L_0 = S$ and $H_n = G$). If we focus on a present hit point H_k, a hit point $H_i (i < k)$ is called a past point, on the other hand, another hit point $H_i (i > k)$ is called a future point. Based on the metric condition, a future [past] point is closer [further] to G than a present point.

In order to evaluate an upper bound of path length, we focus on a set of hit points. In both algorithms, a robot meets and leaves a hit point along its straight segment, clockwise boundary, and counter-clockwise boundary at most once. For this reason, an upper bound of selected paths is calculated by $2D + 2\Sigma_i P_i$.

By the geometric reason, an arbitrary hit point is classified into three types as follows (Fig.5):

a. A mobile robot always finds a leave point by each (clockwise or counter-clockwise) direction from a hit point (Fig.5(a)).

b. A mobile robot always finds a leave point by a direction (one of the clockwise and counter-clockwise directions) from a hit point, but cannot find any leave point by its opposite direction from the hit point (Fig.5(b)).

c. A mobile robot cannot find any leave point by both directions from a hit point (Fig.5(c)).

Furthermore, all cases returning to a hit point are completely classified into the following three, two, one cases occurred from a hit point whose types are **a.**, **b.**, **c.**, respectively. In other words, any other case does not appear for an uncertain environment in *Rev*1 and *Rev*2. For this reason, it is necessary and sufficient for us to calculate the maximum number of tracing obstacle boundary or tracing straight segment at each case **a-1**, **a-2**, **a-3**, **b-1**, **b-2**, **c-1** (Fig.6(a)~(f)).

Moreover, in **(2d)** and **(2e)** of *Rev*1 and *Rev*2, after encountering Q_l, a robot returns to a booked hit point H_j which is the closest to G. In both algorithms, the robot always selects a shorter route between Q_l and H_j. However in this proof, we should note that a robot always selects a route started by the following an obstacle at Q_l.

Concerning to a global loop found by **(2d)**, which is described in Fig.4(b), a mobile robot passes through H_{i-1}, L_{i-1}, H_i, and $Q_l = H_{i-1}$ in the algorithms and their convergence proof. Then, the robot compares a counter-clockwise route (tracing an obstacle from Q_l to H_i) with a clockwise route (via H_{i-1}, L_{i-1}, and H_i). In this case, since the clockwise route is shorter, the algorithm selects the route. However, the counter-clockwise route following an obstacle is always picked up in the proof. On the other hand, concerning to a local loop found by **(2e)**, a robot returns to a booked hit point H_j which is the closest to G. In both algorithms, the robot always selects a shorter route between Q_l and H_j. However in this proof, we should note that a robot always selects a route started by the following an obstacle at Q_l. For example, in Fig.4(c), a mobile robot passes through H_{i-1}, L_{i-1}, H_i, and $Q_l = L_{i-1}$ in the algorithms and their convergence proof. Then, the robot compares a clockwise route (straight going from L_{i-1} to H_i) with a counter-clockwise route (tracing an obstacle from Q_l to H_i). In this case, since the clockwise route is shorter, the algorithm selects the route. However, the counter-clockwise route is always picked up in the proof. By these, a path in the proof is sometimes longer, but its upper bound is still evaluated by $2D + 2\Sigma_i P_i$.

At a hit point H_l whose type is **a.**, we have the following possibilities: After selecting a direction to follow an obstacle, a mobile robot firstly returns to H_l or not. Note that H_3 in Fig.1 appears as the latter case. In the former case, the robot selects its opposite direction following an obstacle at H_l or goes to a past (further) hit point via H_l (Fig.6(a)). In the former case, the robot secondly returns to H_l or not. Note that H_5 in Fig.1 appears as the latter case. In the former case, the robot always goes away from H_l to a past (further) hit point (Fig.6(b)). Finally, even though a mobile robot does not reverse its direction, the robot sometimes returns to H_l via a global loop. As mentioned previously, the robot always reverses its direction to follow an obstacle in the proof (Fig.6(c)).

At a hit point H_l whose type is **b.**, we have the

		Class1		Bug2		Alg		Rev	
		const.	*alter.*	*const.*	*alter.*	1	2	1	2
n_s:(9,9)-n_g:(256,256)	clockwise	6949	long	15018	16243	2865	7811	2396	7392
	c-clockwise	23733	long	28551	16329	8438	9826	2488	7521
n_s:(502,9)-n_g:(256,256)	clockwise	3272	5932	11022	long	3272	7783	3310	3818
	c-clockwise	23733	5886	12370	long	23733	5515	3899	3196
n_s:(502,502)-n_g:(256,256)	clockwise	25653	28092	15027	long	6609	5462	7197	5540
	c-clockwise	5073	25172	6655	long	5073	6655	6611	4074
n_s:(9,502)-n_g:(256,256)	clockwise	21866	long	7057	3377	21866	6057	3197	2904
	c-clockwise	8833	long	14276	3241	8331	12484	2229	3576

Table 1 : Path lengths from four start points n_s to a goal point n_g in *Class*1, *Bug*2, *Alg*1, *Alg*2, *Rev*1 and *Rev*2 in a 2-D uncertain maze.

		Class1		Bug2		Alg		Rev	
		const.	*alter.*	*const.*	*alter.*	1	2	1	2
n_s:(9,9)-n_g:(275,263)	clockwise	long	long	long	11027	27579	22023	10274	12494
	c-clockwise	long	long	long	10690	10668	11628	1137	3494
n_s:(502,9)-n_g:(275,263)	clockwise	long	long	long	long	18368	14498	19616	2320
	c-clockwise	18578	long	15676	long	5356	4029	15892	2939
n_s:(502,502)-n_g:(275,263)	clockwise	long	long	21626	long	21626	20372	22513	14131
	c-clockwise	15566	29486	6027	long	6027	21633	14819	6541
n_s:(9,502)-n_g:(275,263)	clockwise	26384	8080	long	long	6308	5453	3189	2606
	c-clockwise	9735	7960	16769	long	8063	6015	2542	1590

Table 2 : Path lengths from four n_s to one n_g in *Class*1, *Bug*2, *Alg*1, *Alg*2, *Rev*1 and *Rev*2 in another 2-D uncertain maze.

following possibilities: After selecting a direction to follow an obstacle, a mobile robot firstly returns to H_l or not. In the former case, if the robot returns to H_l continuously without finding any leave point (Fig.6(d)), the robot always selects its opposite direction to follow an obstacle at H_l. Then, the robot returns to H_l or not. Note that H_2 and H_4 in Fig.1 appear as the latter case. In the former case, the robot always goes to a past (further) hit point via H_l.

On the other hand, if the robot returns to H_l discontinuously while finding a leave point, the robot selects its opposite direction following an obstacle at H_l (Fig.6(e)) or goes to a past (further) hit point via H_l (Fig.6(a)). In the former case, the robot continuously returns to H_l and then also goes to a past (further) hit point via H_l.

At a hit point H_l whose type is **c.**, we have the following possibility: After selecting a direction to follow an obstacle, a mobile robot returns to H_l continuously and always selects its opposite direction at H_l. Then, the robot also returns to H_l continuously and always goes to a past (further) hit point via H_l (Fig.6(f)).

Theorem 2 (Case a-1) As shown in Fig.6(a), a mobile robot selects one of clockwise and counter-clockwise directions at a hit point H_l whose type is **a.**. Then, the robot goes away from H_l to a past (further) hit point because it is the closest hit point booked in *Hlist* (H_6 and H_7 are this examples in Fig.1). In this case, a mobile robot never returns to the hit point H_l once more.

Proof: Concerning to the reverse procedure, after selecting a direction to follow an obstacle at H_l, a mobile robot goes to a past (further) hit point via H_l as the closest hit point booked in *Hlist*. By the induction of our algorithm, this means any future (closer) hit point including H_l is not booked in *Hlist*. Consequently, a mobile robot cannot return to H_l from any future (closer) hit point by the reverse procedure. Furthermore in the proof of the reverse procedure, a robot cannot leave an obstacle from a past (further) leave point. Therefore, the robot does not return to H_l from any past (further) leave point.

Concerning to the normal procedure, since H_l was already removed from *Hlist*, it is continuously enclosed by clockwise and counter-clockwise local loops around obstacle boundaries (This was described in **(2e)** in *Rev*1 and *Rev*2.). Because of this obstruction based on theorem 1, the robot cannot return to H_l directly or indirectly.

As a result, a mobile robot never returns to the hit point H_l once more. Q.E.D.

Theorem 3 (Case a-2) As shown in Fig.6(b), a mobile robot selects one of clockwise and counter-clockwise directions at a hit point H_l whose type is **a.**. Then, the robot returns to H_l as the closest hit point booked in *Hlist*, and therefore selects its opposite direction (continues to trace) at H_l. Then, the robot returns to H_l again as the closest hit point. Consequently, since both directions were tested at H_l, the robot goes to a past (further) hit point as the closest hit point. In this case, the mobile robot never returns to the hit point H_l once more.

Proof: Concerning to the normal procedure, because of theorem 1, a mobile robot cannot return to H_l directly. Moreover, the robot cannot return to H_l indirectly by straightforward going and obstacle tracing because it is completely protected by three leave points L_p, L_l and L_m. The reason is as follows: If a robot encounters three leave points L_p, L_l and L_m, the robot reverses its direction to follow an obstacle.

Concerning to the reverse procedure, after selecting a direction to follow an obstacle at H_l, a mobile robot returns to H_l as the closest hit point booked in *Hlist*. Therefore, the robot selects its opposite direction at H_l. Moreover, the robot returns to H_l again as the closest hit point booked in *Hlist*. Consequently, both directions were tested at H_l, and therefore it is removed from *Hlist*. Because of the induction of our algorithm, this means no future (closer) hit point is located in *Hlist*. Therefore, a mobile robot cannot return to H_l from any future (closer) hit point by the reverse procedure. Furthermore in the proof of the reverse procedure, a robot cannot leave an obstacle from a past (further) leave point.

Therefore, the robot does not return to the point H_l from any past (further) leave point.

As a result, a mobile robot never returns to the hit point H_l once more. Q.E.D.

Theorem 4 (Case a-3) As shown in Fig.6(c), a mobile robot selects one of clockwise and counter-clockwise directions at a hit point H_l whose type is **a.**. Then, if a robot returns to H_l directly from its opposite direction via a global loop, the robot always reverses its direction to follow an obstacle in the proof (H_1 is this example in Fig.1). Here, we should note that H_l is not removed from *Hlist* because the opposite direction is not tested by the normal procedure. In this case, a mobile robot never returns to the hit point H_l once more.

Proof: Concerning to the normal procedure, because of theorem **1**, a robot cannot arrive at H_l directly or indirectly since H_l is completely blocked from three traced directions by three leave points L_p, L_l and L_m.

Concerning to the reverse procedure, a mobile robot always regards at least the last hit point H_m on a global loop as the closest hit point booked in *Hlist*, unless there is not any deadlock-free path until a destination in an unknown environment. As long as the last hit point H_m is left, the robot always reverses the following direction and consequently reaches H_m by the proof of the reverse procedure. For this reason, the robot never returns to the hit point H_l by the reverse procedure.

As a result, a mobile robot never returns to the hit point H_l once more. Q.E.D.

Theorem 5 (Case b-1) As shown in Fig.6(d), a mobile robot firstly selects a direction to follow an obstacle at a hit point H_l whose type is **b.**. Then, the robot returns to H_l continuously without finding any leave point by the reverse procedure. In this case, the robot always selects the opposite direction at H_l. Then, if the robot returns to H_l, the robot always aims at a past (further) hit point including H_l, which is the closest hit point booked in *Hlist*. In this case, a mobile robot never returns to the hit point H_l once more.

Proof: See the proof in theorem 3. Q.E.D.

Theorem 6 (Case b-2) As shown in Fig.6(e), a mobile robot firstly selects a direction to follow an obstacle at a hit point H_l whose type is **b.**. Then, the robot returns to H_l by the reverse procedure after finding a leave point L_l. In this case, the robot goes to a past (further) hit point or returns to H_l as the closest hit point booked in *Hlist*. Then in the latter case, the robot selects the opposite direction, and always returns to H_l continuously without finding any leave point, and always goes to a past (further) hit point. In this case, a mobile robot never returns to the hit point H_l once more.

Proof: In the former case, see the proof in theorem 2. In the latter case, see the proof in theorem 3. As a result, a mobile robot never returns to the hit point H_l once more. Q.E.D.

Theorem 7 (Case c-1) As shown in Fig.6(f), a robot selects a direction to follow an obstacle at a hit point H_l whose type is **c.**. In this case, the robot continuously returns to H_l via L_k. Then, the robot also selects its opposite direction and then returns to H_l via L_o continuously (Fig.6(f)). In this case, the robot never returns to H_l once more (H_8 is this example in Fig.1).

Proof: See the proof in theorem 3. Q.E.D.

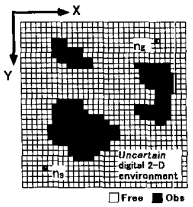

Figure 7 : A cell robot and its 2-D digital map with *Free* and *Obs* cells (nodes).

4 Simulation Results

In this section, we compare paths and lengths selected by typical sensor-based path-planning algorithms *Bug*2, *Class*1, *Alg*1, *Alg*2, *Rev*1 and *Rev*2 in two kinds of uncertain huge mazes.

In this section, we adopt a **cell decomposition approach** to path planning, which has a flexibility for representing a complicated huge maze (Fig.7). First of all, a 2-D uncertain environment is defined by a digital map $[x_m, y_m](m = 1, ..., 2^i)$. All the cells are built by dividing X and Y axes into an arbitrary resolution. S and G are denoted as n_s and n_g, respectively. According to the division, a digital map is built by 4^i cells (nodes). Then, all cells (nodes) are classified into two categories such as *Free* and *Obs* cells (nodes). If and only if a cell (node) does not intersect any obstacle, the cell (node) is *Free*, otherwise, the cell (node) is *Obs*. Secondly, a robot is expressed by a cell (node), and is able to move one of neighbor cells (nodes) in the digital map. In general, eight cells (nodes) are prepared for neighbors. Two neighbor cells (nodes) are implicitly connected by an edge. Furthermore, the distance (cost) of an edge between horizontal and vertical neighbors is defined by one, and the distance (cost) of an edge between diagonal neighbors is defined by a square route two ($\sqrt{2}$). A total distance between start and goal cells (nodes n_s and n_g) is calculated by summing these neighbor distances. In the 2-D digital map [x,y], many obstacles whose shape is complex are colored by gray, and a selected path is colored by black.

Firstly, a huge uncertain maze is randomly constructed, and then three different collision-free paths are selected from n_s:(9,502) to n_g:(256,256) by *Class*1 (*const.*)[*c*−*clockwise*], *Alg*2[*c*−*clockwise*], *Rev*2[*c*−*clockwise*] in an uncertain maze shown in Fig.8(a),(b) and (c), respectively. In addition, three different paths are selected from n_s:(9,9) to n_g:(256,256) by

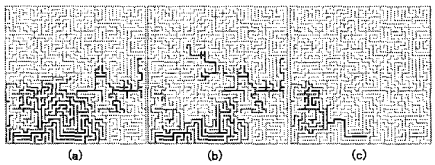

Figure 8 : (a) A collision-free path from n_s:(9,502) to n_g:(256,256) selected by *Class*1(*const.*)[*c* − *clockwise*] in a 2-D uncertain maze. (b) A path between the same pair selected by *Alg*2[*c* − *clockwise*] in the maze. (c) A path between the same pair selected by *Rev*2[*c* − *clockwise*] in the maze.

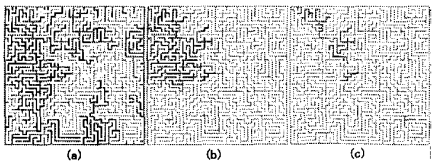

Figure 9 : (a) A collision-free path from n_s:(9,9) to n_g:(256,256) selected by $Bug2(alter.)[c-clockwise]$ in a 2-D uncertain maze. (b) A path between the same pair selected by $Alg1[c-clockwise]$ in the maze. (c) A path between the same pair selected by $Rev1[c-clockwise]$ in the maze.

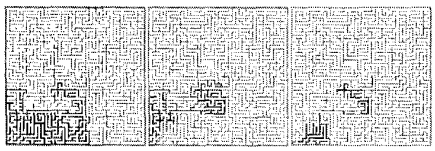

Figure 10 : (a) A collision-free path from n_s:(9,502) to n_g:(275,263) selected by $Class1(alter.)[clockwise]$ in another 2-D unknown maze. (b) A path between the same pair selected by $Alg2[clockwise]$ in the maze. (c) A path between the same pair selected by $Rev2[clockwise]$ in the maze.

$Bug2(alter.)[c-clockwise]$, $Alg1[c-clockwise]$ and $Rev1[c-clockwise]$ in the same maze shown at Fig.9(a), (b) and (c), respectively. Furthermore, if we change n_s, we obtain the other results (Table 1, *long* means path length is over 30000). In the results, our algorithm $Rev1$ or $Rev2$ almost selects the shortest collision-free path until a destination.

Secondly, another huge uncertain maze is built at random and then three different paths are picked up from n_s:(9,502) to n_g:(275,263) by $Class1(alter.)[clockwise]$, $Alg2[clockwise]$, $Rev2[clockwise]$ in another unknown maze illustrated at Fig.10(a),(b) and (c), respectively. In addition, three collision-free paths are selected from n_s:(502,9) to n_g:(275,263) by $Bug2(const.)[c-clockwise]$, $Alg1[c-clockwise]$ and $Rev2[c-clockwise]$ in the same uncertain maze shown at Fig.11(a),(b) and (c), respectively. Furthermore, by changing n_s, we obtain the other results (Table 2, *long* means path length is over 30000). In the results, our algorithms $Rev1$ or $Rev2$ almost selects the shortest collision-free path until a destination.

As a result, on the average, new algorithms $Rev1$ or $Rev2$ catches the shortest or a shorter path in a huge uncertain maze whose shape is complicated. Finally, all programs are running in a personal computer Gateway GP6-400 (Windows 98 with Open GL, Pentium II 400MHz, 256MB main memory).

5 Conclusions

In this paper, we evaluated an upper bound of path lengths in $Rev1$ and $Rev2$ as $2D+2\Sigma_i P_i$ which is slightly larger than that made in $Alg1$ and $Alg2$. Furthermore, we compared typical seven sensor-based path-planning algorithms in two kinds of uncertain mazes, and regarded the algorithm $Rev1$ or $Rev2$ as the best on the average. This was unfortunately ascertained by simulation results, and therefore will be evaluated theoretically in future.

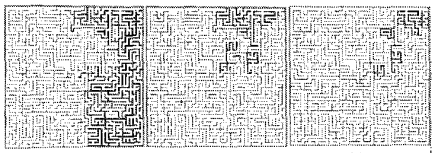

Figure 11 : (a) A collision-free path from n_s:(502,9) to n_g:(275,263) selected by $Bug2(const.)[c-clockwise]$ in another 2-D unknown maze. (b) A path between the same pair selected by $Alg1[c-clockwise]$ in the maze. (c) A path between the same pair selected by $Rev2[c-clockwise]$ in the maze.

References

[1] V.J.Lumelsky and A.A.Stepanov, "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape," *Algorithmica*, Vol.2, pp.403-430, 1987.

[2] A.Sankaranarayanan and M.Vidyasager, "A new path planning algorithm for moving a point object amidst unknown obstacles in a plane, *Proc. of the 1990 IEEE Int. Conf. on Robotics and Automation*, pp.1930-1936, May 1990.

[3] H.Noborio,"A path-planning algorithm for generation of an intuitively reasonable path in an uncertain 2D workspace", *Proc. of the Japan-USA Symposium on Flexible Automation*, pp.477-480, July 1990.

[4] A.Sankaranarayanan and M.Vidyasagar, "Path planning for moving a point object amidst unknown obstacles in a plane: a new algorithm and a general theory for algorithm develop-ment," *Proc. of the 29th Conf. on Decision and Control*, pp.1111-1119, December, 1990.

[5] H.Noborio, "A sufficient condition for designing a family of sensor-based deadlock-free path-planning algorithms," *Journal of Advanced Robotics*, Vol.7, No.5, pp.413-433, January 1993.

[6] H.Noborio, "On a sensor-based navigation for a mobile robot," *Journal of Robotics Mechatronics*, Vol.8, No.1, pp.2-14, 1996.

[7] H.Noborio, T.Yoshioka, and S.Tominaga, "On the sensor-based navigation by changing a direction to follow an encountered obstacle," *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.510-517, September 1997.

[8] S.Sundar and Z.Shiller, "Optimal obstacle avoidance based on the hamilton-jacobi-bellman equation," *IEEE Trans. on Robotics and Automation*, Vol.13, No.2, pp.305-310, 1997.

[9] I.Kamon and E.Rivlin, "Sensory-based motion planning with global proofs," *IEEE Trans. on Robotics and Automation*, Vol.13, No.6, pp.814-822, 1997.

[10] H.Noborio, M.Kadowaki, and K.Urakawa, "A near-optimal sensor-based motion-planning algorithm for parts mating," *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.510-517, October 1998.

[11] H.Noborio, K.Fujimura, Y.Horiuchi, "A comparative study of sensor-based path-planning algorithms in an unknown maze", *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.909-916, October 2000.